Read mapping exercise

Objective

You will use a dataset of Illumina whole-genome sequencing reads from human samples. There are ~20 different datasets to choose from, each identified by a different letter (A, B, C ...) - pick one at random.

These datasets are artificially constructed for this exercise, but conceptually it might be similar to data you might get from some biomedical sequencing experiment. For example, genome screening data from an embryo, or from a tumour sample. Your task is to find out as much as possible about your dataset.

Each dataset differs slightly, in three ways. Find the answer to the following three questions:

Question 1) How much of the sequenced DNA is human? There's some amount of bacterial contamination in every dataset, meaning some of the reads are not of human origin, rather they are of bacterial origin. Find out what fraction of the reads in your dataset are human.

Question 2) Which chromosome is duplicated? Every dataset has a simulated duplication for one of the chromosomes. So instead of there being two copies of that chromosome, there are three copies. This condition is known as a *trisomy*. For example, trisomy of chromosome 21 causes Downs syndrome, and it's also common for cancer tumours to acquire extra copies of chromosomes. Find out what chromosome is duplicated in your dataset.

Question 3) What is the sex chromosome karyotype? Biological females have two X chromosomes (XX karyotype), while biological males have one X and one Y chromosome (XY karyotype). Find out the sex chromosome karyotype in your dataset.

Step 1: Start an interactive Slurm session and load software

Use the following command to start an interactive job where you can run commands without necessarily having to submit Slurm jobs for every command:

```
interactive-bio-ds
```

Before running any commands, ensure that the required software modules are loaded:

```
module load bwa/0.7.17
module load samtools/1.21
```

Check if they are correctly loaded by running the "`bwa`" and "`samtools`" commands without any arguments. If they are loaded, a summary of the software, its version number, and a list of available sub-commands will be displayed. Brtiefly familiarise yourself with sub-commands that are available.

Step 2: Copy your chosen dataset

The sequencing datasets are stored in the following directory (run an "`ls`" command to list them):

```
/gpfs/data/BIO-DSB/Session5/datasets/
```

Pick one of them, and copy the files to your home folder. Important: these are paired-end datasets, so you need to make sure you copy both of the files, both the "_1.fastq.gz" and the "_2.fastq.gz" files. Before you start working, you probably want to create a new directory for this exercise. An example:

```
pwd                 #check which folder you are in
mkdir read-mapping  #creates a new directory
cd read-mapping/    #move into the new folder
cp /gpfs/data/BIO-DSB/Session5/datasets/datasetX_* .   #copy both files
ls -ltrh            #check the files are now here
```

Step 3: Study your input files

Use some basic Unix commands to study your input FASTQ files. The files are compressed with gzip, meaning you can't directly look at them with commands like "head" or "less". But there are tricks we can use to uncompress the file on the fly:

```
zless datasetX_1.fastq.gz           #less for gzipped files. "q" to exit
zcat datasetX_1.fastq.gz | head     #cat for gzipped files, piped into head
zcat datasetX_1.fastq.gz | wc -l    #pipe into wc to count lines
```

How many sequencing reads are in your file? Remember the structure of the FASTQ format. Make sure that the _1 file and _2 files have the same number of lines.

Step 4: Study the reference genome

You can find the human reference genome here:
`/gpfs/data/BIO-DSB/Session5/ref/human-ref-GRCh38.fasta`

Have a look at the file with `head` or `less`. How many chromosomes are in the file? Use the "grep" command to keep only the FASTA header lines (lines starting with the ">" character).

In order to map reads with the BWA software, the reference genome needs to be indexed – meaning it is converted into a data structure that the software can more easily use to align reads to. This indexing process takes a couple of hours, which we don't have time for in this workshop. Therefore, we have already performed the indexing for you – we used the following command:

```
bwa index /gpfs/data/BIO-DSB/Session5/ref/human-ref-GRCh38.fasta
```

The index files are located next to the fasta file, with various strange file extensions:

```
ls -ltrh /gpfs/data/BIO-DSB/Session5/ref/
total 8.0G
-rw-r--r-- 1 rjr22jpu BIO-DSB-LEC 3.0G Jan 28 09:35 human-ref-GRCh38.fasta
-rw-r--r-- 1 rjr22jpu BIO-DSB-LEC  16K Jan 29 14:33 human-ref-GRCh38.fasta.amb
-rw-r--r-- 1 rjr22jpu BIO-DSB-LEC  954 Jan 29 14:33 human-ref-GRCh38.fasta.ann
-rw-r--r-- 1 rjr22jpu BIO-DSB-LEC 2.9G Jan 29 14:33 human-ref-GRCh38.fasta.bwt
-rw-r--r-- 1 rjr22jpu BIO-DSB-LEC  788 Jan 29 14:33 human-ref-GRCh38.fasta.fai
-rw-r--r-- 1 rjr22jpu BIO-DSB-LEC 737M Jan 29 14:34 human-ref-GRCh38.fasta.pac
-rw-r--r-- 1 rjr22jpu BIO-DSB-LEC 1.5G Jan 29 14:34 human-ref-GRCh38.fasta.sa
```

Step 5: Align reads to the reference genome

Use BWA-MEM to map the reads to the reference genome. This process is too memory-intensive to be run in the interactive session, so it needs to be submitted as a separate Slurm job. Here is a template command for running bwa for paired-end input d ata – replace the coloured parts to match your files:

```
bwa mem /path/to/reference.fasta datasetX_1.fastq.gz datasetX_2.fastq.gz > X.sam
```

Then put your command into a Slurm submission script. You can copy the template given in Session 2, which is also provided here:
`/gpfs/data/BIO-DSB/Session5/Example_SLURM_submission_script.sh`

You will need to modify the Slurm submission script to ask for appropriate computational resources. You will probably need at least 10GB of memory, and 10 minutes of runtime. 1 core is enough. You can edit the script directly on the command line using the `nano` text editor (remember: `ctrl+X` to quit).

Use Slurm commands such as `sacct` to track the progress of your job. After job completion, study the `.STDOUT` and `.STDERR` files to understand if the job has finished successfully, or if there are any error messages.

Step 6: Convert SAM to BAM format

First study your output SAM files using Unix tools like less. Tip: use "`less -S`" for more comfortable viewing of files that have very long lines (hit "`q`" to quit `less`). Try to understand what you're looking at!

Use the `samtools view` command to convert your SAM file to BAM format. Find the appropriate argument to output in the BAM format.

Compare the files sizes of the original SAM file and the new BAM file. How much smaller is the BAM file?

Step 7: Sort the BAM file

Use samtools to sort the BAM file by chromosomal coordinate. Run "`samtools`" without any arguments to display the list of available sub-commands, find the samtools sub-command for sorting, and try to understand how to run that command. Output the sorted reads to a new BAM file – think about how to name that new file.

Study your sorted BAM file to verify that the reads are now sorted by chromosomal position. You can't use `less` on a BAM file, as it's compressed. Instead you have to use `samtools view`, but you can pipe the output to less on the fly:

```
samtools view -h file.bam | less -S
```

Step 8: Index the BAM file

Create an index for the sorted BAM file to enable efficient querying. This is done using the "`samtools index`" command, and results in a small file with a "`.bai`" extension. You rarely if ever have to use this file directly yourself, but various software (including samtools itself) can use the index file to speed up data retrieval from the BAM file.

As an example, try the command "`samtools idxstats`" on your BAM file, to very quickly retrieve information on the number of reads that map to each chromosome.

Step 9: Extract mapping statistics

Use the `samtools flagstat` command to extract some basic statistics from the BAM file.

What fraction of reads successfully mapped to the reference genome?

Step 10: Study coverage across chromosomes

Use the `samtools coverage` command to calculate coverage/depth for all the different chromosomes. Note that in the output of this command, the `coverage` column refers to something different than how we used the term "coverage" in the lecture (it refers to the % of bases that are covered). Instead, the most relevant column to look at is the `meandepth` column – this holds the average read coverage, or depth, as discussed in the lecture.

Tip: with a tab-delimited output like this, the columns often get misaligned in the output, making it more difficult to look at. Pipe the output into the command `column -t` to get a nicer display.

Sort the results based on coverage depth in descending order. Can you find what chromosome has been duplicated?

Step 11: Determine sex chromosome karyotype

Using the same `samtools coverage` output, look at the coverage of the X and Y chromosomes. What is the sex chromosome karyotype of your dataset?

Step 12: Estimate mitochondrial copy number

Look at the coverage of the mitochondrial DNA (chrM) relative to nuclear DNA. Why is it much higher? Can you estimate the approximate number of mitochondria per cell based on the coverage values?